Q7. Email, subdomain and DNS reconnaissance

Original question: Perform reconnaissance on an authorized domain to collect publicly available email addresses, subdomains, and DNS information. Record your findings and explain the significance of any two observations.

Aim

Collect publicly indexed email/subdomain leads and DNS records for an approved domain, and explain at least two observations.

Requirements

Approved domain; theHarvester, whois, dig (dnsutils), internet access. Public sources may rate-limit or require API keys. Reference: [R4].

Procedure

Step 1 - Prepare and confirm supported sources

```
mkdir -p "$HOME/EHF_Exam/Q7"
cd "$HOME/EHF_Exam/Q7"
DOMAIN='<AUTHORIZED_DOMAIN>'
date -Is | tee collected_at.txt
theHarvester -h
whois "$DOMAIN" > whois.txt
```

Use the case-sensitive executable theHarvester. Read the source list in the installed help; backend availability changes. Do not assume every online tutorial's search engine still works.

Step 2 - Collect public-source leads

```
theHarvester -d "$DOMAIN" -l 100 -b duckduckgo \
  -f harvest_search > harvest_search.log 2>&1
cat harvest_search.log
theHarvester -d "$DOMAIN" -l 100 -b crtsh \
  -f harvest_certs > harvest_certs.log 2>&1
cat harvest_certs.log
ls -l
```

Use these named backends only if listed in installed help; otherwise replace the backend with a supported approved public source. `-d` selects the domain, `-l` limits search results, `-b` chooses a backend and `-f` saves reports; current Kali documentation describes XML/JSON reports, but inspect actual filenames. The redirected logs also retain errors. Search and certificate records may reveal names, but do not establish current existence. No API brute force, DNS brute force, takeover checks or endpoint scans are used.

Step 3 - Collect DNS details separately

```
dig "$DOMAIN" A > dns_A.txt
dig "$DOMAIN" AAAA > dns_AAAA.txt
dig "$DOMAIN" MX > dns_MX.txt
dig "$DOMAIN" NS > dns_NS.txt
cat dns_A.txt dns_AAAA.txt dns_MX.txt dns_NS.txt
```

DNS lookups contact your resolver; they are a validation step, distinct from strictly passive public indexing. Use the full response to distinguish NOERROR with no answers, NXDOMAIN and SERVFAIL. For a discovered in-scope hostname, run `dig '<DISCOVERED_SUBDOMAIN>' A` after substitution, saving the result. Do not try random guessed hosts.

Expected output - illustrative record

```
Email lead: helpdesk@example.org | source: indexed contact page
Subdomain lead: portal.example.org | source: certificate record
MX: 10 mail.example.org.
```

The examples are synthetic. No emails found is a valid backend result; a backend error is not evidence that none exist. Record the source URL if available, backend, collection time and whether DNS validation succeeded.

Interpretation, observation and result

Maintain separate lists for email leads, subdomain leads and DNS answers. Explain two actual observations: for example, an MX record identifies a configured mail exchanger, not an individual mailbox; a certificate hostname suggests a historical service name, not necessarily a live server. Public email addresses can increase exposure to unsolicited contact, but their publication does not prove compromise. The practical succeeds when the collection method, actual findings or absence, and two supported interpretations are recorded.

Common errors

- Missing API key: select an approved keyless source or configure your own permitted key privately.
- Backend/CAPTCHA/rate-limit error: record failure, avoid bypassing it and try another supported public source.
- NXDOMAIN for a lead: it may be stale, a wildcard certificate entry or a typo; preserve that limitation.

Viva questions

- Is a certificate name proof of a live host? No; certificates may be historical or use wildcards.
- What does MX specify? Mail exchange hosts and their preference values.
- Does a DNS query count as strictly passive collection? No; it sends a query to a resolver/server.
- Why retain source and time? Public records change and findings must be reproducible.